

# **UAB**

## **Universitat Autònoma de Barcelona**

Diseño e implementación de una API de control de acceso para la gestión de usuarios y organitzacions

Informe inicial

Igor Ulyanov Melnic  
Tutora: Victoria Montenegro Ruíz

Grado en Ingeniería Informática  
Escuela de Ingeniería  
Curso 2025-2026

## 0. Control del documento

### 0.1 Información del documento

|                     |                     |
|---------------------|---------------------|
| Identificador       | informeInicial      |
| Autor               | Igor Ulyanov Melnic |
| Fecha de creación   | 17/2/2026           |
| Ultima modificacion | 7/2/2026            |
| Nombre del fichero  | Informe Inicial.pdf |

### 0.2 Historial del documento

| Versión | Fecha     | Cambios                                                |
|---------|-----------|--------------------------------------------------------|
| 1.0     | 17/2/2026 | Creación del documento, entrada de información inicial |
| 1.1     | 22/2/2026 | Esbozo inicial del documento                           |
| 1.2     | 27/2/2026 | Desarrollo de apartados y esquema                      |
| 1.3     | 1/3/2026  | Primera iteración del documento final                  |
| 1.4     | 7/3/2026  | Ajustes finales                                        |

# Índice

|                                      |           |
|--------------------------------------|-----------|
| <b>0. Control del documento.....</b> | <b>2</b>  |
| 0.1 Información del documento.....   | 2         |
| 0.2 Historial del documento.....     | 2         |
| <b>1. Introducción.....</b>          | <b>4</b>  |
| 1.1 Resumen.....                     | 4         |
| 1.2 Palabras clave.....              | 4         |
| <b>2. Metodología.....</b>           | <b>5</b>  |
| <b>3. Motivación.....</b>            | <b>6</b>  |
| <b>4. Alcance y Objetivos.....</b>   | <b>7</b>  |
| 4.1 Alcance.....                     | 7         |
| 4.2 Objetivos.....                   | 8         |
| <b>5. Planificación.....</b>         | <b>9</b>  |
| <b>6. Uso de IA generativa.....</b>  | <b>11</b> |
| <b>7. Bibliografía.....</b>          | <b>11</b> |

# 1. Introducción

## 1.1 Resumen

El presente Trabajo de Fin de Grado aborda el diseño e implementación de un servicio de control de acceso corporativo expuesto mediante una API REST<sup>[1]</sup>. El sistema tiene como finalidad centralizar la gestión de usuarios, organizaciones, roles y permisos, permitiendo a aplicaciones externas consultar si un usuario determinado está autorizado a realizar una acción concreta dentro de un contexto organizacional específico. Esta aplicación puede servir, por ejemplo, como un módulo desacoplado de un sistema empresarial más grande, al cual otras aplicaciones acceden para verificar permisos.

La solución se fundamenta en un modelo de autorización basado en RBAC<sup>[2]</sup> (Role-Based Access Control), en el que los permisos se asignan a roles y estos, a su vez, se vinculan a usuarios dentro de organizaciones determinadas. Este enfoque permite separar claramente la lógica de autenticación de la lógica de autorización, favoreciendo la modularidad y la escalabilidad del sistema. La autenticación se implementará mediante un mecanismo stateless basado en JSON Web Tokens (JWT)<sup>[3]</sup>, lo que permitirá la identificación segura de los usuarios sin necesidad de mantener un el servidor.

El backend se desarrollará utilizando Java<sup>[4]</sup> y el framework Spring<sup>[5]</sup>, aprovechando su ecosistema para la construcción de APIs REST seguras y mantenibles. La persistencia de datos se realizará sobre una base de datos relacional PostgreSQL<sup>[6]</sup>, diseñada para soportar relaciones complejas entre usuarios, organizaciones y roles. El sistema será desplegado en un entorno contenerizado mediante Docker, garantizando portabilidad y facilidad de despliegue.

El proyecto contempla la implementación de un motor de autorización capaz de evaluar dinámicamente los permisos asociados a un usuario en función de su rol dentro de una organización concreta. Asimismo, se desarrollarán funcionalidades CRUD (Create, Read, Update and Delete) para la gestión de los distintos elementos del modelo, así como documentación automática de la API mediante especificación Swagger<sup>[7]</sup>.

En conjunto, el trabajo pretende desarrollar una solución estructurada y alineada con prácticas habituales en entornos empresariales, aplicando principios de seguridad, separación de responsabilidades y diseño modular.

## 1.2 Palabras clave

API, desarrollo, autorizacion, organización, backend

## 2. Metodología

La metodología de desarrollo adoptada seguirá un enfoque principalmente en cascada, incorporando ciertos elementos organizativos propios de metodologías ágiles con el fin de mejorar la planificación y el seguimiento del progreso. Este planteamiento me permitirá estructurar el proyecto en fases claramente diferenciadas, manteniendo al mismo tiempo herramientas de control y adaptación durante la implementación.

En una primera etapa se llevará a cabo la fase de ideación y recogida de requisitos, donde se definirán las necesidades que deberá cubrir la aplicación a nivel funcional. A partir de esta fase se elaborará el correspondiente documento de requisitos, que servirá como base formal para el desarrollo posterior. Este primer paso marcará la pauta para el resto del desarrollo pero me mantengo abierto a la posibilidad de cambios, debido a que tengo poco conocimiento previo de las herramientas y tecnologías en uso.

Acto seguido, se abordará la fase de diseño, en la que se definirán los principales componentes del sistema. En esta etapa se generará documentación propia, incluyendo el diagrama de la arquitectura y el esquema de la base de datos, con el objetivo de establecer una visión clara y estructurada del funcionamiento interno de la aplicación.

De forma complementaria, se organizará el trabajo mediante la creación de un backlog que recogerá las tareas necesarias para la implementación de los requisitos definidos. El seguimiento del progreso se realizará con un tablero Kanban a través de la herramienta Trello, lo que permitirá visualizar el estado de las tareas y mantener un control continuo del avance del proyecto. Este tablero me permitirá tener un barómetro visual del progreso del proyecto además de cierta trazabilidad. Más allá de las 3 etapas clave del tablero (tareas planificadas, en progreso y hechas) añadiré columnas de problemas detectados, problemas resueltos y tareas descartadas. Estas columnas son las que me darán trazabilidad para mis acciones durante el desarrollo.

Una vez el sistema y trabajo a realizar están definidos se procederá a la fase de implementación, donde se desarrollarán los elementos incluidos en el backlog. Este proceso será acompañado con documentos de seguimiento que recojan la evolución del trabajo, así como las dificultades encontradas y las soluciones adoptadas. Esta práctica facilitará la trazabilidad del desarrollo y la justificación de las decisiones técnicas tomadas.

Para concluir, se pasará a una fase de revisión, pulido y cierre del trabajo, asegurando la coherencia global del proyecto y la calidad del resultado final.

### 3. Motivación

La motivación principal para la realización de este trabajo surge del interés por ampliar conocimientos hacia el ámbito del desarrollo backend y la creación de APIs. Este proyecto representa una oportunidad para complementar mis conocimientos existentes de frontend para obtener una visión más global del desarrollo de aplicaciones.

Asimismo, existe una base previa en el lenguaje Java, lo que motiva a experimentar con frameworks populares del ecosistema y profundizar en su uso dentro de un contexto real. Esta exploración permitirá consolidar conocimientos y adquirir experiencia práctica en tecnologías que se usan en la actualidad.

Otro aspecto relevante de la motivación es el interés por aprender a crear aplicaciones mediante Docker, con el objetivo de facilitar su despliegue, portabilidad y mantenimiento. Del mismo modo, el proyecto permitirá trabajar con bases de datos relacionales como PostgreSQL, reforzando competencias en modelado, persistencia de datos e integración con APIs.

Además, se pretende profundizar en el diseño de APIs, en las buenas prácticas de documentación y en la comprensión de estándares utilizados en entornos profesionales. Este aprendizaje contribuirá a mejorar la calidad del software desarrollado y a facilitar su uso por terceros.

Finalmente, el proyecto se plantea con la intención de crear una herramienta open-source que pueda resultar útil en un entorno real. Esta orientación práctica no solo incrementa la motivación personal, sino que también permite trabajar con criterios profesionales, priorizando la reutilización, la mantenibilidad y la posible adopción por parte de la comunidad.

## 4. Alcance y Objetivos

### 4.1 Alcance

La API que se deberá desarrollar constituirá el núcleo funcional del sistema y será desplegada en un entorno contenerizado compuesto por dos contenedores independientes: uno destinado a la aplicación backend y otro a la base de datos relacional.

La solución se desarrollará utilizando Java y SpringBoot, integrando un sistema de autenticación stateless basado en JSON Web Tokens (JWT). El sistema diferenciará claramente entre los procesos de autenticación (validación de credenciales y emisión de token) y autorización (verificación de permisos asociados a roles dentro de una organización). La principal razón por la cual he escogido Java + SpringBoot es debido a que es un tech stack popular a nivel empresarial y profesional, además cuenta con una robusta comunidad y recursos para aprendizaje.

La base de datos relacional, basada en PostgreSQL, almacenará la información necesaria para la gestión de usuarios, organizaciones, roles y permisos, así como las relaciones entre estos elementos. El modelo de datos se diseñará siguiendo el paradigma RBAC (Role-Based Access Control), permitiendo la asignación de roles a usuarios dentro de organizaciones específicas y la asociación de permisos a dichos roles. Una base de datos relacional me permitirá mantener una estructura rígida de los datos, esto a su vez facilitará la búsqueda de la información necesaria para realizar el servicio propuesto.

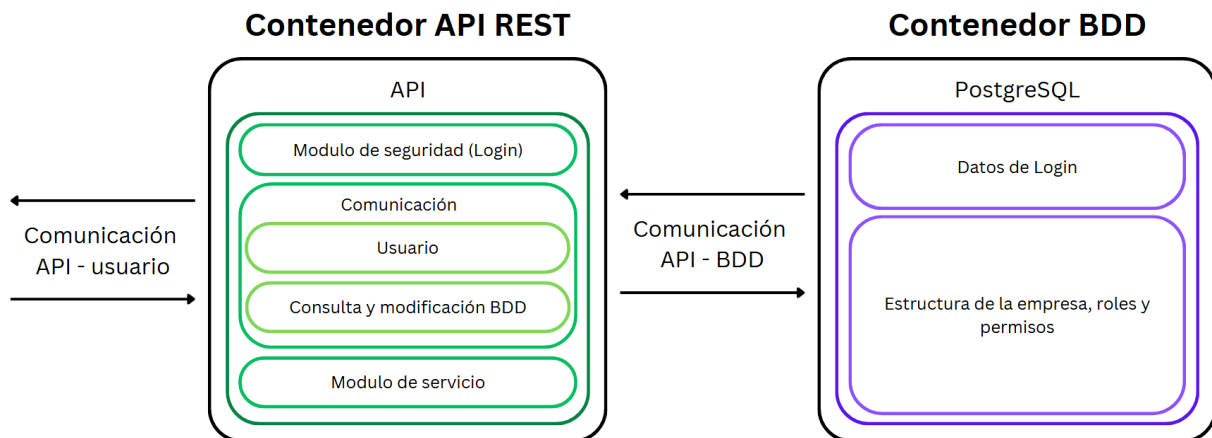
El sistema incorporará un motor de autorización que permitirá validar si un usuario dispone de los permisos necesarios para ejecutar una acción concreta dentro del contexto de una organización determinada. Este servicio estará desacoplado de la lógica de negocio de aplicaciones externas, actuando como un módulo centralizado de control de acceso.

Se permitirá que un mismo usuario pueda pertenecer a múltiples organizaciones y desempeñar distintos roles en cada una de ellas. Los usuarios, permisos y organizaciones se manejan por un usuario del sistema con el rol MASTER. Este será un rol de sistema con privilegios globales, independiente de las organizaciones, encargado de la administración general del sistema.

Asimismo, el proyecto incluirá:

- Funcionalidades CRUD para la gestión de usuarios, organizaciones y roles.
- Definición de un conjunto de permisos predefinidos.
- Mecanismos de control de acceso diferenciados entre roles de sistema (por ejemplo, rol "MASTER") y roles organizacionales.
- Documentación de la API mediante especificación Swagger.
- Despliegue simplificado a través de contenedores Docker.

Como elemento adicional de carácter opcional, se contempla el desarrollo de un frontend sencillo que permita demostrar el funcionamiento práctico del sistema.



*Fig. 1: Diagrama de componentes inicial*

## 4.2 Objetivos

Los objetivos específicos del proyecto son los siguientes:

- Implementar un sistema de autenticación basado en JWT que permita la identificación segura de los usuarios.
- Diseñar e implementar un modelo de autorización basado en RBAC que permita asignar roles a usuarios dentro de organizaciones específicas.
- Desarrollar un motor de autorización capaz de determinar si un usuario puede ejecutar una acción concreta sobre un recurso determinado.
- Implementar un sistema CRUD completo para la gestión de usuarios, organizaciones, roles y permisos.
- Establecer la figura de un usuario con rol de sistema "MASTER", encargado de la creación de nuevas organizaciones y usuarios, así como de la asignación inicial de contraseñas.
- Permitir que los usuarios puedan modificar su propia contraseña una vez autenticados.
- Diseñar el modelo de datos optimizando las relaciones críticas e incorporando índices que garanticen tiempos de respuesta adecuados en las consultas de autorización.
- Documentar la API de manera clara y estructurada.
- Facilitar el despliegue y la reproducibilidad del sistema mediante un entorno contenerizado basado en Docker.



En conjunto, estos objetivos buscan desarrollar una solución funcional, estructurada y alineada con prácticas habituales en entornos empresariales, aplicando principios de seguridad, modularidad, mantenibilidad y escalabilidad.

## 5. Planificación

El presente diagrama de Gantt tiene como objetivo representar la planificación temporal del desarrollo del proyecto. En él, las tareas marcadas en color verde corresponden a actividades de documentación y diseño; las tareas en color magenta representan el desarrollo de código; y los rombos amarillos señalan las fechas de entrega de los distintos artefactos del trabajo.

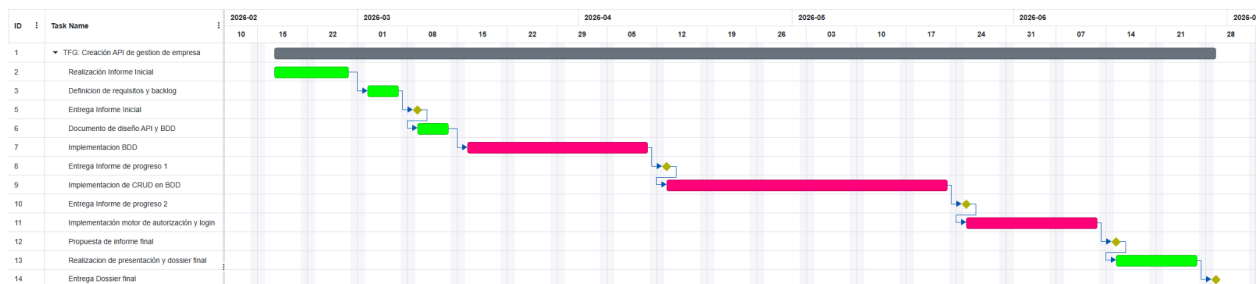


Fig. 2: Diagrama de Gantt representando la planificación realizado con Online Gantt [1]

El proyecto se iniciará con una fase de ideación, en la que se elaborará el informe inicial y se realizará una primera definición de los requisitos, así como del backlog de tareas que estructurará la fase posterior. Este backlog se irá ajustando conforme evolucionen las necesidades del proyecto, procurando mantenerlo siempre alineado con la idea principal. La fase concluirá una semana después de la entrega del informe inicial e incluirá también la elaboración de un diseño preliminar tanto de la API como del esquema de la base de datos relacional. La duración estimada de esta etapa es de aproximadamente cuatro semanas.

La siguiente fase es de desarrollo, tiene una duración estimada de trece semanas y constituye el núcleo de la implementación del sistema. Se organiza en varias subfases con el objetivo de garantizar un progreso sistemático.

### 1. Configuración del entorno y dependencias (Semana 1)

En esta etapa se preparará el entorno de desarrollo, asegurando que todas las herramientas y dependencias estén correctamente configuradas. Las tareas incluyen:

- Configuración del proyecto Java con Spring Boot.
- Integración de Spring Security para la autenticación y autorización.
- Configuración de PostgreSQL en un contenedor Docker independiente, con volúmenes persistentes.
- Preparación de herramientas de documentación automática (Swagger).

### 2. Implementación de la capa de persistencia y modelo de datos (Semanas 2–4)

Se definirá e implementará el esquema de base de datos y la capa de acceso a datos. Las tareas incluyen:

- Definición de esquema relacional
- Implementación de índices y restricciones para optimizar las consultas críticas de autorización.
- Inserción de datos iniciales de prueba, incluyendo roles, permisos y el usuario MASTER.

### **3. Desarrollo de la capa de negocio (Semanas 5–8)**

En esta subfase se implementará la lógica de negocio que soporta la autenticación, autorización y gestión de entidades. Las tareas incluyen:

- Servicios para la gestión CRUD de usuarios, roles, organizaciones y permisos.
- Implementación del motor de autorización que valida permisos por rol y organización.
- Gestión segura de contraseñas mediante hashing
- Integración de JWT para la autenticación stateless.

### **4. Desarrollo de la capa de exposición y API REST (Semanas 9–10)**

Se expondrá la funcionalidad mediante endpoints REST y se documentará la API. Las tareas incluyen:

- Implementación de controladores REST para usuarios, roles, organizaciones y permisos
- Configuración de Swagger para documentación automática.
- Pruebas de endpoints simulando distintos escenarios de autorización.

### **5. Pruebas y depuración (Semanas 11–12)**

Se garantizará el correcto funcionamiento del sistema, la integridad de los datos y la seguridad de la API. Las tareas incluyen:

- Pruebas de servicios y repositorios.
- Simulación de distintos usuarios y roles para validar permisos.
- Optimización de consultas críticas y resolución de errores detectados.

### **6. Preparación del informe final y artefactos de entrega (Semana 13)**

Se consolidará la documentación y los resultados del desarrollo. Las tareas incluyen:

- Redacción de la memoria final del TFG.
- Actualización de la documentación de despliegue mediante Docker.
- Preparación de ejemplos de uso
- Integración opcional del frontend demostrativo

Finalmente, antes de la entrega del dossier definitivo, se reservarán dos semanas adicionales para la preparación de la presentación, la elaboración del póster y la revisión y formalización final del trabajo.

## 6. Uso de IA generativa

La inteligencia artificial se utilizará en este trabajo como una herramienta de apoyo al aprendizaje. Debido a que será necesario trabajar con tecnologías nuevas para mí, su función principal será ayudar en la comprensión de conceptos desconocidos y la exploración de alternativas. Asimismo, servirá para identificar posibles fuentes y orientar la búsqueda de la información necesaria, sin constituir en ningún caso la fuente principal.

En este contexto, la herramienta actuará como un medio para facilitar el acceso a recursos adecuados y para ayudar a digerir y organizar la información obtenida, pero no sustituirá el proceso de investigación propio. El desarrollo del código, la selección de fuentes y la interpretación de la información serán responsabilidad propia.

Además, la inteligencia artificial se empleará como apoyo en la redacción de la documentación, especialmente en la corrección ortográfica y en el retoque del formato. Su uso estará orientado al pulido y mejora de textos ya elaborados, excluyendo la generación directa del contenido final.

De este modo, la información presentada refleja mi progreso y comprensión personal. La IA generativa se utilizará únicamente para agilizar tareas de revisión, permitiendo dedicar más tiempo al desarrollo del trabajo.

## 7. Bibliografía

- [1] Richardson, L., & Amundsen, M. (2013). *RESTful Web APIs*. O'Reilly Media.
- [2] NIST. (2012). *Role-Based Access Control (RBAC)*. National Institute of Standards and Technology. <https://csrc.nist.gov/projects/role-based-access-control>
- [3] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- [4] Oracle. (2023). *The Java Language Specification, Java SE 21 Edition*. Oracle Corporation. <https://docs.oracle.com/javase/specs/>
- [5] VMware. (2023). *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [6] PostgreSQL Global Development Group. (2023). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/>
- [7] SmartBear. (s.f.). *Swagger: API design and documentation tools*. Recuperado el 4 de marzo de 2026, de <https://swagger.io/>
- [8] Online Gantt. (s.f.). *Free online Gantt chart software*. Recuperado el 1 de marzo de 2026, de <https://www.onlinegantt.com/>